

TitanZero — Database Doctrine & Schema Model(Revised)

Purpose

This document defines the structural rules governing the TitanZero database.

The doctrine ensures that:

every record belongs to a tenant

operational entities are modular and extensible

lifecycle events are traceable

signals drive automation and AI reasoning

the schema remains consistent across engines and extensions

This doctrine must be followed whenever new tables are created.

Naming Doctrine

TitanZero uses three table classes.

tz_* → core operational data

ext_* → extension module data

(no prefix) → platform system tables

Platform Tables (No Prefix)

These represent global platform infrastructure.

Examples:

users

teams

roles

permissions

plans

menus
extensions
openai

These tables are not tenant-owned.

They define the platform environment.

Core Operational Tables (tz_)

Tables prefixed with tz_ represent tenant-owned business data.

Examples:

tz_customers
tz_jobs
tz_quotes
tz_invoices
tz_evidence
tz_signals

Rules:

must include team_id

must represent business activity

must follow lifecycle entity pattern

Extension Tables (ext_)

Tables prefixed with ext_ belong to optional extensions.

Examples:

ext_social_accounts
ext_campaigns

ext_marketing_messages

Rules:

must include team_id

must not break tenant isolation

must integrate via signals rather than core modifications

Tenant Ownership Doctrine

Every operational record belongs to a company.

The tenant boundary is defined by:

team_id

The system also keeps:

company_id

user_id

For compatibility with existing modules.

Meaning

Column	Purpose
team_id	primary tenant boundary
company_id	compatibility mirror
user_id	responsible actor

All operational tables must contain:

team_id

created_at

updated_at

Entity–Satellite Pattern

TitanZero entities follow a core table + satellite tables pattern.

Instead of storing everything in one table, the entity is separated into modular components.

Example: Jobs

tz_jobs
tz_jobs_events
tz_jobs_notes
tz_jobs_evidence
tz_jobs_links

Core Entity Table

The main entity table stores stable lifecycle state.

Example:

tz_jobs

Fields:

id
team_id
customer_id
assigned_user_id
status
scheduled_at
completed_at
created_at
updated_at

This table contains only the current state of the entity.

Events Table

Tracks lifecycle changes.

Example:

tz_jobs_events

Fields:

id
team_id
job_id
event_type
event_data
created_by
created_at

Examples:

job.created
job.started
job.completed
job.cancelled

Events form the historical timeline of the entity.

Notes Table

Stores human or AI notes attached to an entity.

Example:

tz_jobs_notes

Fields:

id
team_id
job_id

note
created_by
created_at

Used by:

staff notes

AI lifecycle assistants

quality control annotations

Evidence Table

Stores files and proof records.

Example:

tz_jobs_evidence

Fields:

id
team_id
job_id
file_path
evidence_type
uploaded_by
created_at

Evidence types may include:

photo
video
document
inspection
compliance

Link Table

Allows entities to connect to other system objects.

Example:

tz_jobs_links

Fields:

id
team_id
job_id
target_type
target_id
created_at

Example links:

job → invoice
job → inspection
job → conversation
job → campaign

Signal System Tables

Signals drive automation and AI reasoning.

Signals are stored in:

tz_signals

Structure:

id
team_id
event_type
entity_type
entity_id
payload

created_at

Example events:

job.created
quote.accepted
invoice.sent
invoice.paid
campaign.launched

Signals are consumed by:

Titan Pulse

Lifecycle assistants

Titan Zero governance engine

Automation Tables

Automation rules and executions are stored separately.

tz_automation_rules
tz_automation_runs

Example rule:

id
team_id
trigger_event
conditions
actions
priority
created_at

Automation runs record:

rule_id
signal_id
execution_result

created_at

Communication Tables

Used by Titan Talk.

tz_conversations

tz_messages

Messages may be linked to entities through link tables.

Example:

conversation → job

conversation → customer

conversation → invoice

Quality Control Tables

Used by Titan Go QC Mode.

Example tables:

tz_inspections

tz_inspection_items

tz_qc_reports

Fields support:

inspection results

compliance checks

evidence attachment

QC approval workflow

Lifecycle Signals

Lifecycle transitions automatically generate signals.

Examples:

quote.sent
quote.accepted
job.started
job.completed
invoice.sent
invoice.paid
campaign.launches

These signals trigger:

automation rules

lifecycle assistant responses

Titan Zero governance evaluation

Schema Expansion Rules

When developers add tables, the following rules must be followed.

1. Operational tables must include team_id.
2. Lifecycle entities must follow entity–satellite pattern.
3. Events must generate signals.
4. Extensions must use ext_ prefix.

5. Platform tables must not use prefixes.

6. Tables must support auditability through timestamps.

Summary

The TitanZero database doctrine ensures:

strict tenant ownership

modular lifecycle entities

event-driven automation

scalable schema expansion

compatibility with AI reasoning engines

The entity–satellite pattern prevents operational tables from becoming oversized and allows new capabilities to be added without breaking existing structures.